Chapter 2: Programming a guess game

Here's how it works: the program will generate a random integer between 1 and 100.
It will then prompt the player to enter a guess. After a guess is entered, the program will indicate whether the guess is too low or too high.
If the guess is correct, the game will print a congratulatory message and exit.

Setting up a new project

Create a new cargo project using "cargo new guessing_game --bin" where bin referes at the directory as binary (executable);

In main.rs we inlcude into the scope the '::io' library, ask for inputs using the macro println!(), declare a mutable variabile and inside of that we allocate a new string using the 'new()' mehtod on inside the String:: class.

Then we call the standard input 'stdin()' we use the method 'read_line()' on it passing the mutable reference of the 'guess' variabile.
The 'read_line()' method is accepting a &reference as parameter and outputting a 'io::Result' which is an enum containing Ok and Err: Err contains information about how or why the operation has failed.

If an instance 'of io::Result' is an Err value, 'expect' will cause the programe to crash and display the message that you passed as argument to that 'expect'

Generating a Secret Number

We dont have a random function in std::, so we'll have to import it under 'dependencies' in the .toml file using:
' rand = "0.3.14" '

You can use 'cargo update' to update every dependency or use the cargo.lock to recreate each time the same exact artifact.

So now we added:

- extern crate rand;
- use rand::Rng;
- let secret_number = rand::thread_rng().gen_range(1,101);

Comparing the Guess with the Secret Number

To use the compare features we need to add them to the scope from the std:: , using the cmp() method and importing the Ordering type (enum).

Ordering is another enum but the variants inside it are 'Less', 'Greater', 'Equal'.

So now we added:

- use std::cmp::Ordering;
- match guess.cmp(&secret_number) { Ordering::Less => println!("Wrong: too small!"); Ordering::Greater => println!("Wrong: too big!"); Ordering:::Equal => println!("You win!") }

The cmp() method compares two values and can be called on anything that can be compared, it takes as parameter the &reference about what is it comparing to.

We use the 'match' expression to decide what to do next based on which variant of Ordering was returned from the cmp() call.
'match' is made up of arms: arms consists of a pattern and the code that should be run if the values given to the

beginning of the match expression fits that arm's pattern. (More about it in chapter 6 and 18). \

As for now the program doesn't compile because of a type mismatch while comparing the declared Sting type of the input and the numerical (i32/u32/i64/etc…) of the secret_number.

So now we add:

- let guess: u32 = guess.trim() .parse() .expect("Please type a number!")

We shadowed delcared guess as u32 and binding it the '.trim()' and 'parse()' methods;
Shadowing lets us reuse the guess variable name rather than forcing us to create two unique variables, such as guess_str and guess, for example.
The 'trim()' method on a String instance will eliminate any whitespace at the beginning and end.
The 'parse()' method on strings parses a string into some kind of number.

Because the 'parse()' method can easily cause errors we want to also return a Result type as we did before using the 'expect()' method.

Allowing multiple guessing

Using the 'loop' keyword we can create an infinte loop.

Quitting after a correct guess

We can just edit the Ordering::Equal behavior by adding a 'break' keyword;

Handling invalid input

We can also refine the input method by defining the correct behavior upon a non-number user input;

To do so we can edit the 'guess' definition by replacing the '.expect()' function to a 'match' expression;
Switching from an expect call to a match expression is how you generally move from crashing on an error to handling the error. Remember that parse returns a Result type and Result is an enum that has the variants Ok or Err.